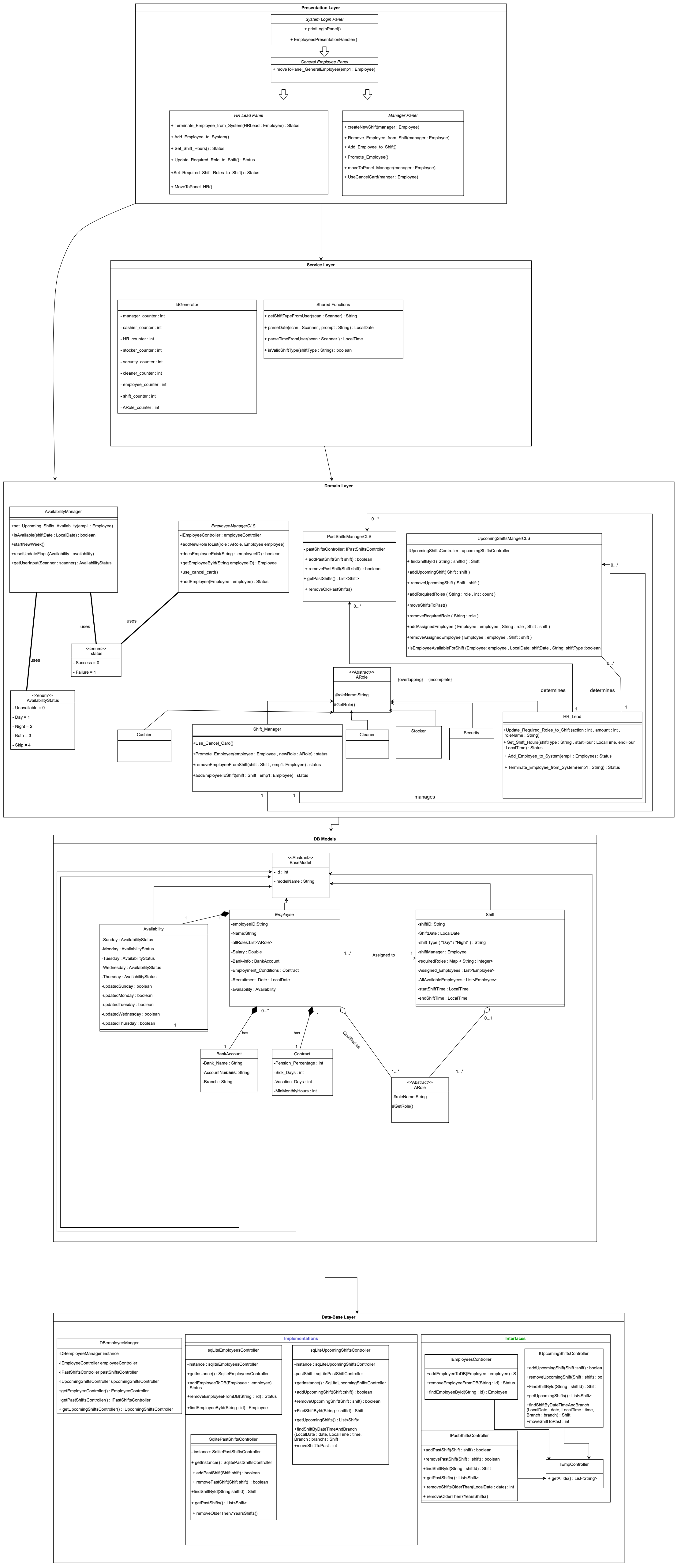
Presentation Layer
System Login Panel
+ printLoginPanel()
+ EmployeesPresentationHandler()
General Employee Panel
+ moveToPanel_GeneralEmployee(emp1 : Employee)
HR Lead Panel
+ Terminate_Employee_from_System(HRLead : Employee) : Status
+ Add_Employee_to_System()
+ Set_Shift_Hours() : Status
+ Update_Required_Role_to_Shift() : Status
+Set_Required_Shift_Roles_to_Shift() : Status
+ MoveToPanel_HR()
Manager Panel
+ createNewShift(manager : Employee)
+ Remove_Employee_from_Shift(manager : Employee)
+ Add_Employee_to_Shift()
+ Promote_Employee()
+ moveToPanel_Manager(manager : Employee)
+ UseCancelCard(manger : Employee)
Service Layer
IdGenerator
- manager_counter : int
- cashier_counter : int
- HR_counter : int
- stocker_counter : int
- security_counter : int
- cleaner_counter : int
- employee_counter : int
- shift_counter : int
- ARole_counter : int
Shared Functions
+ getShiftTypeFromUser(scan : Scanner) : String
+ parseDate(scan : Scanner , prompt : String) : LocalDate
+ parseTimeFromUser(scan : Scanner) : LocalTime
+ isValidShiftType(shiftType : String) : boolean
Domain Layer
AvailabilityManager
+set_Upcoming_Shifts_Availability(emp1 : Employee)
+isAvailable(shiftDate : LocalDate) : boolean
+startNewWeek()
+resetUpdateFlags(Availability : availability)
+getUserInput(Scanner : scanner) : AvailabilityStatus
EmployeeManagerCLS
-IEmployeeController : employeeController
+addNewRoleToList(role : ARole, Employee employee)
+doesEmployeeExist(String : employeeID) : boolean
+getEmployeeById(String employeeID) : Employee
+use_cancel_card()
+addEmployee(Employee : employee) : Status
uses
uses
uses
<<enum>>
status
- Success = 0
- Failure = 1
<<enum>>
AvailabilityStatus
- Unavailable = 0
- Day = 1
- Night = 2
- Both = 3
- Skip = 4
0...*
PastShiftsManagerCLS
- pastShiftsController: IPastShiftsController
+ addPastShift(Shift shift) : boolean
+ removePastShift(Shift shift) : boolean
+ getPastShifts() : List<Shift>
+ removeOldPastShifts()
0...*
UpcomingShiftsMangerCLS
-IUpcomingShiftsController : upcomingShiftsController
+ findShiftById (String : shiftId) : Shift
+addUpcomingShift(Shift : shift)
+ removeUpcomingShift (Shift : shift)
+addRequiredRoles (String : role , int : count)
+moveShiftsToPast()
+removeRequiredRole (String : role)
+addAssignedEmployee (Employee : employee , String : role , Shift : shift)
+removeAssignedEmployee (Employee : employee , Shift : shift)
+isEmployeeAvailableForShift (Employee: employee , LocalDate: shiftDate , String: shiftType :boolean
0...*
0...*
<<Abstract>>
ARole
#roleName:String
#GetRole()
{overlapping}
{incomplete}
determines
determines
1
1
Cashier
Shift_Manager
+Use_Cancel_Card()
+Promote_Employee(employee : Employee , newRole : ARole) : status
+removeEmployeeFromShift(shift : Shift , emp1: Employee) : status
+addEmployeeToShift(shift : Shift , emp1: Employee) : status
Cleaner
Stocker
Security
HR_Lead
+Update_Required_Roles_to_Shift (action : int , amount : int , roleName : String)
+ Set_Shift_Hours(shiftType : String , startHour : LocalTime, endHour : LocalTime) : Status
+ Add_Employee_to_System(emp1 : Employee) : Status
+ Terminate_Employee_from_System(emp1 : String) : Status
1
1
manages
DB Models
<<Abstract>>
BaseModel
- id : Int
- modelName : String
Availability
-Sunday : AvailabilityStatus
-Monday : AvailabilityStatus
-Tuesday : AvailabilityStatus
-Wednesday : AvailabilityStatus
-Thursday : AvailabilityStatus
-updatedSunday : boolean
-updatedMonday : boolean
-updatedTuesday : boolean
-updatedWednesday : boolean
-updatedThursday : boolean
1
1
1
Employee
-employeeID:String
-Name:String
-allRoles:List<ARole>
-Salary : Double
-Bank-info : BankAccount
-Employment_Conditions : Contract
-Recruitment_Date : LocalDate
-availability : Availability
1...*
Assigned to
1
Shift
-shiftID: String
-ShiftDate : LocalDate
-shift Type ("Day" / "Night") : String
-shiftManager : Employee
-requiredRoles : Map < String : Integer>
-Assigned_Employees : List<Employee>
-AllAvailableEmployees : List<Employee>
-startShiftTime : LocalTime
-endShiftTime : LocalTime
0...*
1
has
has
Qualified as
0...1
1
1
BankAccount
-Bank_Name : String
-AccountNumber : String
-Branch : String
uses
Contract
-Pension_Percentage : int
-Sick_Days : int
-Vacation_Days : int
-MinMonthlyHours : int
1...*
1...*
<<Abstract>>
ARole
#roleName:String
#GetRole()
Data-Base Layer
DBemployeeManger
-DBemployeeManager instance
-IEmployeeController employeeController
-IPastShiftsController pastShiftsController
-IUpcomingShiftsController upcomingShiftsController
+getEmployeeController() : EmployeeController
+getPastShiftsController() : IPastShiftsController
+ getUpcomingShiftsController() : IUpcomingShiftsController
Implementations
sqLiteEmployeesController
-instance : sqliteEmployeesController
+getInstance() : SqliteEmployeesController
+addEmployeeToDB(Employee : employee) : Status
+removeEmployeeFromDB(String : id) : Status
+findEmployeeById(String : id) : Employee
sqLiteUpcomingShiftsController
-instance : sqLiteUpcomingShiftsController
-pastShift : sqLitePastShiftController
+getInstance() : SqLiteUpcomingShiftsController
+addUpcomingShift(Shift :shift) : boolean
+removeUpcomingShift(Shift : shift) : boolean
+FindShiftById(String : shiftId) : Shift
+getUpcomingShifts() : List<Shift>
+findShiftByDateTimeAndBranch (LocalDate : date, LocalTime : time, Branch : branch) : Shift
+moveShiftToPast : int
SqlitePastShiftsController
- instance: SqlitePastShiftsController
+ getInstance() : SqlitePastShiftsController
+ addPastShift(Shift shift) : boolean
+ removePastShift(Shift shift) : boolean
+findShiftById(String shiftId) : Shift
+ getPastShifts() : List<Shift>
+ removeOlderThen7YearsShifts()
Interfaces
IEmployeesController
+addEmployeeToDB(Employee : employee) : S
+removeEmployeeFromDB(String : id) : Status
+findEmployeeById(String : id) : Employee
IUpcomingShiftsController
+addUpcomingShift(Shift :shift) : boolea
+removeUpcomingShift(Shift : shift) : bo
+FindShiftById(String : shiftId) : Shift
+getUpcomingShifts() : List<Shift>
+findShiftByDateTimeAndBranch (LocalDate : date, LocalTime : time, Branch : branch) : Shift
+moveShiftToPast : int
IPastShiftsController
+addPastShift(Shift : shift) : boolean
+removePastShift(Shift : shift) : boolean
+findShiftById(String : shiftId) : Shift
+ getPastShifts() : List<Shift>
+ removeShiftsOlderThan(LocalDate : date) : int
+ removeOlderThen7YearsShifts()
IEmpController
+ getAllIds() : List<String>